



Basics of Docker

A friendly, practical guide for someone learning Docker for the first or second time

From "what is it?" → running a multi-service app

1 Why Docker Exists

🤖 **The problem it solves: "It works on my machine."**

Software depends on a lot of stuff around it — a specific language version, libraries, system packages, environment variables, and config. When those differ between your laptop, a teammate's laptop, and the production server, things break in confusing, hard-to-debug ways.

Docker's answer: package your app *together with everything it needs to run* into a single unit called a **container**. That container then runs the same way everywhere — your machine, your teammate's, and the server.

What Docker actually gives you

Benefit	What it means
Consistency	The container behaves identically everywhere. No more environment mismatches.
Isolation	Each app runs in its own sandbox. App A needing Python 3.9 and App B needing Python 3.12 don't fight.

Easy setup	A new teammate runs one command instead of spending a day installing dependencies.
Lightweight	Containers share the host's OS kernel, so they're far smaller and faster to start than full virtual machines.
Reproducibility	The Dockerfile is a written, version-controlled recipe for the environment.
Deployment	The same image you tested locally is exactly what runs in production.

2 The Three Core Words

Almost everything in Docker comes back to these three ideas. Learn them and the rest clicks into place.

 Dockerfile The recipe — step-by-step instructions to build the environment.	 Image The packaged result of that recipe — a frozen snapshot.	 Container A running instance of an image — the actual live app.
--	--	--

The shipping-container analogy

It doesn't matter what's inside the box — every ship, crane, and truck handles the standardized container the same way. Docker does that for software: a uniform box the whole system knows how to move and run.

3 What "Sandbox" Means



The playground sandbox

A child's sandbox is a boxed-off area where they can dig and make a mess — but the mess stays *inside the box*. It never spills onto the rest of the yard.

In software, a **sandbox** is the same idea: an isolated, walled-off space where a program runs **without affecting anything outside it**. Each Docker container gets its own private version of the things it needs:

Inside the sandbox, the app sees...	...but it's isolated, so
Its own filesystem	It can't see or touch your real files or another app's files
Its own libraries & versions	App A's Python 3.9 doesn't clash with App B's Python 3.12
Its own network ports & processes	One app crashing doesn't take down the others
Its own memory space	A bug in one container can't corrupt another



Why this matters

Safety — if an app misbehaves or gets hacked, the damage is contained in its box. **No conflicts** — incompatible versions run side by side, each unaware of the other. **Clean** — delete the container and everything inside goes with it; nothing is left installed on your real machine.

Your Computer

```
|— 📦 Container A (sandbox) → its own files, libraries, network
|— 📦 Container B (sandbox) → its own files, libraries, network
|— 📦 Container C (sandbox) → its own files, libraries, network
    ↑ each box is walled off from the others and from your real system
```

4 The Real Win: Many Services Together

Docker shines most when you have **multiple services that must run together and talk to each other**. Imagine a real app made of four pieces:

Service	Its job
LiveKit	Real-time audio / video calling
Redis	Caching — fast in-memory storage
Backend	Your API / business logic
Frontend	The UI users actually see

Without Docker you'd install and start each one by hand, in the right order, with the right config, hoping versions don't clash. **Docker Compose** turns all of that into one file and one command.

How it helps

- **One command starts everything** — describe all services in a `docker-compose.yml`, then run `docker compose up` and they all boot together.
- **They find each other by name** — your backend connects to Redis at `redis://redis:6379`, where `redis` is just the service's name, not an IP. Docker wires up an internal network so names "just work."
- **No version conflicts** — each service lives in its own sandbox with its own dependencies.
- **Reproducible for the team** — a new dev clones the repo, runs one command, and gets the exact same stack.

What the Compose file roughly looks like

```
services:
  redis:
    image: redis:7
    ports:
      - "6379:6379"

  livekit:
    image: livekit/livekit-server
    ports:
```

```

    - "7880:7880"
  command: --dev

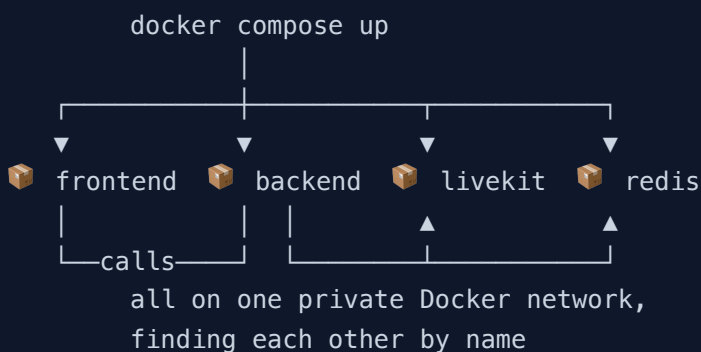
backend:
  build: ./backend          # your own code
  ports:
    - "8000:8000"
  environment:
    REDIS_URL: redis://redis:6379    # talk to Redis by name
    LIVEKIT_URL: ws://livekit:7880   # talk to LiveKit by name
  depends_on:
    - redis
    - livekit

frontend:
  build: ./frontend        # your own code
  ports:
    - "3000:3000"
  depends_on:
    - backend

```

🔑 Things to notice in that file

image: pull a ready-made service (Redis, LiveKit) from the internet • **build:** build *your own* code from its Dockerfile • **depends_on:** controls startup order • **environment:** pass config like connection URLs • **ports:** expose a service to your browser (`"3000:3000"`)



⚠️ One real-world caveat: LiveKit

Real-time media (LiveKit) needs many UDP ports open and correct IP configuration, so it's the one service in this stack that needs extra care inside Docker. It works well — just expect a bit more setup than Redis or a plain API.

- **Docker** packages an app with everything it needs so it runs the same everywhere.
- **Dockerfile** → **Image** → **Container** = recipe → snapshot → running app.
- **Sandbox** = an isolated box; apps can't interfere with each other or your real system.
- **Docker Compose** runs many services (frontend, backend, Redis, LiveKit) together with one file and one command, on a shared private network where they find each other by name.